

Multithreading

1. Create a thread by extending the `Thread` class

Question:

Write a Java program to create a thread by extending the `Thread` class. In the `run()` method, print the thread name and count from 1 to 5.

2. Create a thread by implementing the `Runnable` interface

Question:

Write a Java program to create a thread by implementing the `Runnable` interface. The thread should print "Runnable is running..." five times with a 1-second delay between prints.

3. Use `sleep()` to delay thread execution

Question:

Write a program where two threads print numbers from 1 to 5 with a `Thread.sleep(1000)` delay between each number. Observe the interleaving of output.

4. Set different priorities for threads

Question:

Create three threads named High, Medium, and Low. Set their priorities to `Thread.MAX_PRIORITY`, `Thread.NORM_PRIORITY`, and `Thread.MIN_PRIORITY` respectively. Observe the execution order and explain why it might not always reflect priority settings.

5. Use a lambda expression to create a `Runnable`

Question:

Write a Java program using a lambda expression to create and run a thread that prints "Lambda thread running..." five times with a short delay.

6. Create multiple threads accessing a shared counter (without synchronization)

Question:

Write a program where 3 threads increment a shared counter variable 1000 times. Print the final value. (You should observe inconsistency due to lack of synchronization.)

7. Make shared data thread-safe using `synchronized`

Question:

Modify the previous program by synchronizing the counter increment method to ensure thread safety. Print the final count and verify it's correct.

8. Synchronize a block instead of the whole method

Question:

Write a program where multiple threads write to a shared resource (like a `List<Integer>`). Synchronize only the block where the list is updated, not the entire method.

9. Create a thread-safe bank account using `synchronized`

Question:

Design a simple bank account class with `deposit()` and `withdraw()` methods. Use multiple threads to perform deposits and withdrawals. Use `synchronized` to ensure consistency of the account balance.

10. Demonstrate a deadlock scenario

Question:

Write a Java program to create two threads, each trying to lock two shared resources (e.g., `resource1` and `resource2`) in different orders. Simulate a deadlock, and explain how it occurs.